

NEXUS Project Implementation Plan

Phase 1: CVE and Patch Inventory for Networking UAFs

- **Scan Recent CVEs in Netfilter/Subsystems:** Search CVE databases and security advisories for “use-after-free” or “double-free” in netfilter/nftables and related network code. Key examples include CVE-2026-23231 and CVE-2026-23111 (both nf_tables UAF bugs) [17L194-L202] [37L25-L34] , CVE-2024-1086 (nf_tables UAF) [40L261-L270] , and CVE-2024-26809 (nftables double-free) [13L43-L51] . Also note CVE-2023-32233 (nf_tables UAF) [15L125-L133] . For each CVE, record the affected kernel versions, root causes, and patches (Linux kernel commits).
- **Categorize by Root Cause:** For each identified CVE, determine if it’s due to reference-count errors or lifecycle mishandling. For example, CVE-2026-23111 is caused by an inverted conditional in `nft_map_catchall_activate()` , dropping a chain’s use-count to zero and leaving a stale reference [37L25-L34] . CVE-2026-23231 is a missing `synchronize_rcu()` in `nf_tables_addchain()` ’s error path [17L204-L213] [27L19-L22] . CVE-2024-26809 is a logic flaw in `nft_set_pipapo_destroy()` leading to double-free [13L43-L51] .
- **Identify Reachability:** Determine which bugs are reachable via unprivileged user namespaces. Many nf_tables bugs (e.g. CVE-2026-23111, CVE-2024-26809, CVE-2026-23231) require CAP_NET_ADMIN but can be triggered inside a user+network namespace by mapping UID 0 and using netlink [40L261-L270] [37L25-L34] . Gather evidence (e.g. advisory notes) that these bugs are exploited via unprivileged containers [37L23-L25] [40L261-L270] .
- **Document Vulnerability Details:** For each CVE, summarize its mechanics and root cause (e.g. missing RCU grace period, incorrect refcount decrement) and cite sources or commit links. Include references to authoritative sources (NVD, research blogs, distro advisories). For instance, Exodus Intel’s analysis of CVE-2026-23111 explains the refcount error in detail [37L25-L34] [21L30-L38] . These will guide our exploit development and patch strategy.

Phase 2: Reproducible QEMU Test Environment

- **Select Kernel Version and Configuration:** Choose a baseline Linux kernel version that contains the unpatched vulnerabilities (e.g. latest 6.x mainline before fixes). Clone the kernel source (`git clone git://kernel.org/...`) and check out an appropriate tag or commit [17L39-L48] . Configure the kernel with minimal options: use `make x86_64_defconfig` (for QEMU) and enable necessary features (`CONFIG_USER_NS` , `CONFIG_NET_NS` , netfilter features like `CONFIG_NETFILTER` , `CONFIG_NF_TABLES`) [17L64-L72] [40L261-L270] .
- **Build Custom Kernel:** Install build dependencies (`gcc` , `make` , kernel headers, etc.) [17L58-L67] . Compile the kernel (`make -jN bzImage`). This yields a bootable kernel image (e.g. `arch/x86/boot/bzImage`) [17L72-L81] .
- **Create Minimal Root Filesystem:** Build or provision a minimal userspace. Options include:
 - Using **Buildroot** to compile a tiny Linux FS with Busybox. As described by Offlinemark, one can configure Buildroot for x86_64, set an ext4 image, and use a rootfs overlay to add any custom tools [17L108-L117] [17L124-L134] .
 - Or using **debootstrap** to install a minimal Ubuntu image into a QEMU disk (as per Rainbolt), then install a kernel and packages manually [19L12-L20] . This yields a lightweight Ubuntu root.

- **Assemble QEMU VM:** Install QEMU (`qemu-system-x86_64`) 【17†L91-L94】 . Create a disk image (e.g. via `qemu-img`) and partition it if using debootstrap 【18†L72-L81】 【19†L12-L20】 . Alternatively use Buildroot's ext4 image.
- **Boot and Test:** Launch QEMU with the custom kernel and rootfs (e.g. `qemu-system-x86_64 -kernel bzImage -hda rootfs.ext4 -append "root=/dev/sda rw console=ttyS0" -serial stdio`). Confirm you get a shell. Verify essential commands (`bash` , `ip` , `ping` , etc.). Ensure **user namespaces** are allowed (`kernel.unprivileged_userns_clone=1` or default), since we need unprivileged containers 【40†L303-L310】 .
- **Enable Debugging Aids:** Build a kernel with debug options: enable `CONFIG_KASAN` or at least `CONFIG_SLAB_FREELIST_HARDENED` , `CONFIG_DEBUG_KMEMLEAK` , and dynamic trace options (`FTRACE`) for later exploit tracing. This lets us catch UAFs (KASAN) and inspect heap behavior.

Phase 3: Exploit Development and Validation

- **Map Namespace for Access:** In the QEMU guest, write or use a small C program to `unshare(CLONE_NEWUSER|CLONE_NEWNET)` and map the container's UID 0 to some host UID (via `/proc/[pid]/uid_map`). This gives `CAP_NET_ADMIN` in the new namespace. Alternatively, use `unshare --user --net` with `newuidmap` / `newgidmap` . This mimics an unprivileged container gaining network-capability 【40†L261-L270】 .
- **Trigger Vulnerability via Netlink/NFT:** From inside the namespace, open a netlink socket to netfilter (`socket(AF_NETLINK, SOCK_RAW, NETLINK_NETFILTER)`) or use `libnftnl` . Follow the known trigger pattern: for `nf_tables` bugs, create a table/chain/rule designed to hit the flaw (e.g. create a chain with a catchall verdict, then abort it to hit CVE-2026-23111 【3†L25-L34】). For CVE-2026-23231, attempt to add a chain with IPv4 hook then force IPv6 hook registration to fail 【1†L204-L213】 . Adjust the attack based on the specific vulnerability.
- **Heap Grooming:** Develop a heap spraying strategy to reclaim the freed object. For UAFs, allocate kernel objects of the same size to be placed at the freed address 【24†L29-L37】 . For example, if a freed `nft_chain` was size X, spray with a controlled object (e.g. messages, or special socket objects) of size X. Popov's analysis shows that `kmallocc` will return recently freed addresses unless mitigations are present 【24†L29-L37】 . Loop allocations/freeing to align objects on the same slab and at the target address.
- **Overwrite Kernel Data:** Once a freed object is reallocated with attacker data, use the UAF to overwrite important fields. The ultimate goal is to escalate privileges: overwrite the process's `struct cred` . In Linux, a task's credentials (UIDs/GIDs and capabilities) are stored in `struct cred` pointed by `task->cred` 【33†L84-L92】 【32†L249-L268】 . For example, set `cred->uid=0` , `cred->cap_permitted=all` , etc. Precisely determine the offset of these fields and write them. In practice, one often triggers a kernel function (like `nft_do_chain()`) to dereference the freed pointer, using the overwritten payload to write into the `cred` .
- **Validate Root Escalation:** After exploitation, the container process should have UID 0 (and all capabilities) on the host. Verify by printing `getuid()` and checking `/proc/self/status` fields (or running `id`). If correct, the container has become root on the host. Repeat the exploit multiple times to ensure reliability (the spec expects ~100% success before mitigation).
- **Tracing and Logs:** Use `ftrace` or `dmesg` to trace kernel function calls and verify the exploit path. Enable dynamic tracepoints around the vulnerable functions (`nft_map_catchall_activate` , etc.) to see when refcounts drop to zero. Use `dmesg -w` for any oops or messages. This helps confirm the exact path (for interview detail on "root cause").

Phase 4: Mitigation & Hardening Implementation

- **Patch the Root Cause:** Write and apply a kernel patch fixing the identified bug. For a reference-count logic error (e.g. CVE-2026-23111), revert the inverted condition so the refcount logic is correct, as per the upstream commit [3†L25-L34] [4†L25-L34] . For CVE-2026-23231 (missing RCU), insert `synchronize_rcu()` or adjust the cleanup so it waits before freeing, matching the recommended fix [1†L204-L213] [2†L19-L22] . Compile and boot this patched kernel. Use KASAN or manual tests to confirm the UAF no longer occurs.
- **Allocator Hardening:** Beyond patching, introduce broader defenses. Consider enabling or backporting features like **SLAB_QUARANTINE** or **KFENCE**. Popov's study shows that a slab quarantine prevents immediate reuse of freed objects [24†L47-L55] [24†L47-L54] , breaking heap-spray assumptions. Alternatively enable **CONFIG_RANDOM_KMALLOC_CACHES** (Linux 6.6+), which uses multiple randomized slab caches to thwart predictable allocation [27†L7-L15] . On 6.x kernels, examine a similar defense (e.g. slub freelist hardening).
- **Credential Protection:** If possible, segregate sensitive structures. For example, ensure `struct cred` allocations use a dedicated kmem cache (`cred_jar`) with `SLAB_DESTROY_BY_RCU`, as modern kernels do, to isolate them [30†L25-L33] . If writing a new cache for nftables objects, use `kmem_cache_create()` so that freed chains won't collide with other allocations easily.
- **Restrict Namespace Access (Optional):** As a proactive hardening, disable or tighten unprivileged usersns usage. For example, require additional capabilities for NFTNL netlink in usersns (patch nfnetlink to check `capable(CAP_NET_ADMIN)` even in newnetns). Alternatively, add a kernel config or sysctl to forbid `nft_new_table/chain` without host root, as the Edera analysis notes that netfilter ops should be blocked in usersns if bug-free operation is uncertain [40†L261-L270] [40†L303-L310] .
- **Upstream Submission:** Prepare the patches according to kernel coding style. Test them thoroughly. When validated, submit upstream patches (e.g. via LKML or netfilter mailing list). For interview, be ready to show the diff in the patched code and explain how it fixes the UAF.

Phase 5: Evaluation & Benchmarking

- **Exploit Success Rate:** Measure how often the exploit succeeds before vs. after hardening. Automate repeated exploit runs in both kernels. Record success count; expect ~100% on the baseline (as noted in research) and ~0% on the fixed kernel [40†L283-L290] . Document this "Exploit Success Rate" metric (e.g. "100% → 0%").
- **Performance Impact:** Quantify overhead of the mitigation. Use microbenchmarks for CPU and network:
- **Scheduler (hackbench):** Run the standard hackbench stress test (process-pair ping-pong) to gauge CPU scheduling overhead [35†L9-L12] . Compare latencies in patched vs. original kernel.
- **Network (netperf or iperf):** Measure packet throughput/latency between host and VM or between VMs to assess any slowdown in networking code due to hardened netfilter. For instance, run TCP_STREAM and UDP_STREAM tests with and without nftables rules.
- **Syscall Overhead:** Optionally, measure calls into netfilter (e.g. netlink round-trip time or a simple `getpid()` loop) to see if `synchronize_rcu()` or extra checks add latency.
- **Attack-Surface Metric:** Enumerate available syscalls and netfilter commands before/after. For example, list which NFNL commands succeed without CAP_NET_ADMIN in usersns. Ideally, show that post-hardening, either usersns cannot create certain tables/chains or fewer nf_tables calls succeed. If disabling usersns for netfilter, measure reduction in allowed ioctl/syscalls.

- **Benchmark Results:** Collect data in tables/graphs. For each metric, cite baseline vs. hardened values. If overhead is negligible (e.g. <1-2% extra latency), note that. If measurable, quantify the trade-off. These results form concrete “performance vs. security” claims for your resume.

Each phase’s documentation should cite relevant sources: CVE reports and analyses 【1†L204-L213】 【3†L25-L34】 【13†L43-L51】 , kernel documentation on allocators and cred 【24†L29-L37】 【33†L84-L92】 , and container security studies 【40†L261-L270】 【40†L283-L290】 . Following this plan rigorously will yield a complete NEXUS research project: from real-world kernel bugs to a working exploit, to novel mitigations with measured impacts.
